MedPharm ERP

Medical & Pharmaceutical Management System

Complete Technical Reference Documentation

v1.1.0

Multi-platform enterprise resource planning system for medical practices with integrated pharmaceutical database and insurance claims management.

Platforms: Android | iOS | macOS | Windows | Qt6 Desktop | Flask Web Portal
Stack: Python 3.10+ | SQLAlchemy 2.0 | Flask REST API | Kotlin | SwiftUI | .NET 8

Document Generated: April 16, 2026 at 19:30

Cloud REST API
JWT + CORS

SQLAlchemy DB
75+ Medications

Multi-Platform
6 Client Apps

HIPAA-Aware Design | Role-Based Access | Drug Interaction Safety

Table of Contents

1. System Architecture Overview

MedPharm ERP is a multi-platform medical enterprise resource planning system featuring native clients for Android, iOS, macOS, and Windows, a PyQt6 desktop application for clinical staff, a Flask web portal for patients, and a cloud REST API backend. The architecture follows a clean separation of concerns with the database layer acting as the single source of truth, consumed by a Flask REST API (for mobile/desktop clients) and directly by the PyQt6 desktop and Flask web portal.

1.1 Multi-Platform Architecture

Tier	Technology	Purpose	Users
Presentation (Desktop)	PyQt6 + matplotlib	Full clinical ERP with 9 modules, analytics dashboards, drug interaction checker	Doctors, Psychiatrists, Pharmacists, Admins
Presentation (Web)	Flask + Bootstrap 5 + Jinja2	Patient self-service portal with prescription viewing, bill pay, records	Patients
Cloud API	Flask + JWT + CORS	REST API for mobile/desktop clients with token-based authentication	Android, iOS, macOS, Windows clients
Mobile Clients	Kotlin (Android) SwiftUI (iOS/macOS)	Native patient portals with billing, insurance claims, Rx refills	Patients
Windows Desktop	.NET 8 / WPF	WPF patient portal with DPAPI encrypted token storage	Patients
Data	SQLAlchemy 2.0 + SQLite	23 relational models, ORM layer, 75+ medications, 30+ symptoms, 25+ conditions	All (via managers)

Table 1.1: Three-tier architecture breakdown

1.2 Data Flow

The Qt desktop app, Flask web portal, and cloud REST API all share a single `DatabaseManager` object that wraps all database operations. The manager uses SQLAlchemy's session-per-request pattern via a context manager, ensuring automatic commit on success and rollback on failure. All data is returned as Python dictionaries, creating a natural serialization boundary. Mobile and desktop clients (Android, iOS, macOS, Windows) consume data via the REST API's JSON endpoints.

> **Key Design Decision:** Returning dicts instead of ORM objects from `DatabaseManager` prevents detached-instance errors and makes the data layer frontend-agnostic. The same manager serves both PyQt6 widgets and Flask routes.

1.3 Technology Stack & References

Component	Version	Documentation URL

Python	3.10+	docs.python.org/3
SQLAlchemy	2.0+	docs.sqlalchemy.org/en/20
Flask	3.0+	flask.palletsprojects.com
flask-cors	4.0+	flask-cors.readthedocs.io
PyQt6	6.6+	PyQt6 Reference Guide
Kotlin	1.9+	kotlinlang.org/docs
Retrofit	2.9+	square.github.io/retrofit
SwiftUI	5.0+	Apple SwiftUI Docs
.NET / WPF	8.0	Microsoft WPF Docs
Bootstrap	5.3	getbootstrap.com/docs/5.3
matplotlib	3.8+	matplotlib.org/stable
gunicorn	21.2+	docs.gunicorn.org
Werkzeug	3.0+	werkzeug.palletsprojects.com
SQLite	3.x	sqlite.org/docs.html
ReportLab	4.0+	docs.reportlab.com

Table 1.2: Technology stack with documentation links (clickable)

2. Project Structure & File Organization

```
MedPharm/
├── __init__.py                    # Package root
├── install.sh                     # Automated installer (Bash)
├── run_qt.py                      # Qt desktop launcher
├── run_web.py                     # Flask web launcher
├── run_cloud.py                   # Cloud API server launcher
├── requirements.txt               # Desktop + web dependencies
├── requirements-cloud.txt         # Cloud API dependencies
├── Dockerfile                     # Docker image (API, Ubuntu 24.04)
├── docker-compose.yml             # Docker Compose — API (build from source)
├── docker-compose.hub.yml         # Docker Compose — API (pull from Docker Hub)
├── start_docker_hub.sh            # Interactive launcher for Docker Hub images
├── .dockerignore                  # Docker build exclusions
├── LICENSE                        # GNU GPL v3.0
│
├── .github/workflows/
│   └── docker-publish.yml         # CI/CD auto-build to Docker Hub
│
├── server/                        # ── Docker Server Package ───────
│   ├── Dockerfile                 # Full stack (Ubuntu 24.04 LTS)
│   ├── docker-compose.yml         # Full stack — build from source
│   ├── docker-compose.hub.yml     # Full stack — pull from Docker Hub
│   ├── entrypoint.sh              # DB init & process bootstrap
│   ├── supervisord.conf           # Process manager config
│   ├── nginx/medpharm.conf        # Nginx reverse proxy
│   └── .env.example               # Environment variable template
│
├── database/                      # ── Data Layer ──────────────────
│   ├── models.py                  # 23 SQLAlchemy ORM models + enums
│   ├── db_manager.py              # All CRUD operations (~1100 lines)
│   ├── seed_data.py               # 55 medications, sample data
│   └── seed_expanded.py           # 30+ symptoms, 25+ conditions
│
├── api/                           # ── Cloud REST API ──────────────
│   ├── app.py                     # Flask app factory (CORS, JWT)
│   ├── auth.py                    # HMAC-SHA256 JWT authentication
│   └── routes.py                  # REST endpoints (/api/v1/*)
│
├── qt_app/                        # ── Desktop Application ─────────
│   ├── main_window.py             # QMainWindow + sidebar navigation
│   ├── styles.py                  # 400+ line Qt stylesheet
│   ├── widgets/                   # 9 feature widgets
│   │   ├── billing_widget.py      # Invoices + insurance claims
│   │   ├── symptoms_widget.py     # Symptoms & conditions browser
│   │   └── ...                    # dashboard, patient, rx, etc.
│
├── web/                           # ── Web Portal ──────────────────
│   ├── app.py                     # Flask application factory
│   ├── routes.py                  # All HTTP routes
│   └── templates/                 # 14 Jinja2 HTML templates
│
├── android/                       # ── Android (Kotlin MVVM) ───────
│   └── app/src/main/java/         # Retrofit, OkHttp, Coroutines
│
└── ios/                           # ── iOS (SwiftUI) ───────────────
```

```
■   ■■■ MedPharm/                  # Async/await, Keychain
■
■■■ macos/                         # ■■ macOS (SwiftUI) ■■■■■■■■■■■■
■   ■■■ MedPharm/                  # NavigationSplitView
■
■■■ windows/                       # ■■ Windows (.NET 8 / WPF) ■■■■■■
■   ■■■ MedPharm/                  # DPAPI, CommunityToolkit.Mvvm
■
■■■ docs/
    ■■■ generate_pdf.py            # This documentation generator
```

The project follows a modular package layout. The Python subsystems (`database`, `api`, `qt_app`, `web`) are self-contained and depend only on the `database` package for data access. Native clients (Android, iOS, macOS, Windows) communicate via the `api` REST layer. The web portal can run without PyQt6 installed, and the desktop app can run without Flask.

Total codebase: ~18,000+ lines across 160+ files (Python, Kotlin, Swift, C#/XAML, HTML, CSS, JavaScript, Shell, Nginx, Docker)

3. Database Layer — Models & Schema

The database schema is defined in `database/models.py` using SQLAlchemy 2.0's declarative mapping. All 20 models inherit from a shared `Base = declarative_base()` and use Python enums for type-safe column values. The schema is designed around medical workflow: patients receive diagnoses, which lead to prescriptions containing medication items, which generate invoices paid through the portal.

3.1 Entity Relationship Summary

Entity	Key Fields	Relationships	Purpose
User	username, role, license_number	→ Vitals, Diagnoses, Prescriptions, Appointments, Records, AuditLog	Staff authentication and role-based access
Patient	name, dob, ssn_last4, blood_type, insurance	→ Allergies, Vitals, Diagnoses, Records, Prescriptions, Appointments, Invoices	Central patient record with full medical profile
Medication	ndc_code, brand/generic, schedule, drug_class	→ Interactions (self-ref), PrescriptionItems	55 real medications with FDA data and pricing
MedicationInteraction	medication_a/b_id, severity, description	← Medication (both sides)	24 drug-drug interactions with clinical descriptions
Prescription	rx_number, status, prescribed_date	→ PrescriptionItems ← Patient, User, Diagnosis	Prescription orders with multiple medication items
PrescriptionItem	dosage, frequency, quantity, refills	← Prescription, Medication → InvoiceItems	Individual medication line items with pricing
Invoice	invoice_number, status, total, balance_due	→ InvoiceItems, Payments ← Patient	Billing with line items and payment tracking
Appointment	scheduled_datetime, type, status, duration	← Patient, User	Scheduling with 6 status transitions

Table 3.1: Core entity relationships (8 of 20 models shown)

3.2 Enum Types for Type Safety

The schema uses 16 Python enums stored as string values in SQLite. This approach provides compile-time safety in Python while maintaining human-readable database values. Key enums:

Enum	Values	Used By
UserRole	doctor, psychiatrist, pharmacist, admin	User.role
DrugSchedule	none, II, III, IV, V	Medication.schedule
DrugForm	tablet, capsule, liquid, injection, inhaler, patch, cream, drops, suppository	Medication.form

Enum	Values	Used By
InteractionSeverity	minor, moderate, major, contraindicated	MedicationInteraction
PrescriptionStatus	pending, active, filled, expired, cancelled	Prescription.status
AppointmentStatus	scheduled, confirmed, in_progress, completed, cancelled, no_show	Appointment.status
InvoiceStatus	draft, sent, partial, paid, overdue, cancelled	Invoice.status
AllergySeverity	mild, moderate, severe, life_threatening	Allergy.severity

Table 3.2: Type-safe enum definitions

3.3 Model Definition Pattern

Each model follows a consistent pattern: primary key, foreign keys, data columns with appropriate types, enum-typed status fields, timestamps, and relationship declarations. Here is the Medication model as a representative example:

```
class Medication(Base):
    __tablename__ = "medications"

    id              = Column(Integer, primary_key=True, autoincrement=True)
    ndc_code        = Column(String(20), unique=True, index=True)       # FDA NDC
identifier
    brand_name      = Column(String(200), nullable=False)
    generic_name    = Column(String(200), nullable=False, index=True)   # Indexed for
search
    manufacturer    = Column(String(200))
    drug_class      = Column(String(200), nullable=False, index=True)   # e.g.
"SSRI", "Beta Blocker"
    schedule        = Column(Enum(DrugSchedule), default=DrugSchedule.NONE)  # DEA
schedule
    route           = Column(Enum(DrugRoute), default=DrugRoute.ORAL)
    form            = Column(Enum(DrugForm), default=DrugForm.TABLET)
    strength        = Column(String(50))                                # e.g.
"500mg", "10mg/5ml"
    avg_wholesale_price = Column(Numeric(10, 2))                        # AWP
pricing
    retail_price    = Column(Numeric(10, 2))                            #
Patient-facing price
    is_controlled   = Column(Boolean, default=False)                    # Derived
from schedule

    # Relationships: self-referencing many-to-many for drug interactions
    interactions_as_a = relationship("MedicationInteraction",
        foreign_keys="MedicationInteraction.medication_a_id",
back_populates="medication_a")
    interactions_as_b = relationship("MedicationInteraction",
        foreign_keys="MedicationInteraction.medication_b_id",
back_populates="medication_b")
```

SQLAlchemy Reference: Declarative Mapping Guide — covers Column types, relationship(), and back_populates patterns used throughout.

4. Database Layer — DatabaseManager Operations

The `DatabaseManager` class in `database/db_manager.py` (~600 lines) is the sole interface between the application and the database. It handles connection management, session lifecycle, password hashing, and all CRUD operations. Both the Qt and Flask frontends call these same methods.

4.1 Session Management Pattern

```
class DatabaseManager:
    def __init__(self, db_path: str):
        self.engine = None
        self._session_factory = None

    def init_db(self):
        self.engine = create_engine(
            f"sqlite:///{self.db_path}",
            connect_args={"check_same_thread": False}  # Required for SQLite +
threads
        )
        Base.metadata.create_all(self.engine)           # Auto-create all tables
        self._session_factory = sessionmaker(bind=self.engine)

    @contextmanager
    def get_session(self):
        session = self._session_factory()
        try:
            yield session
            session.commit()    # Auto-commit on clean exit
        except Exception:
            session.rollback()  # Auto-rollback on any exception
            raise
        finally:
            session.close()     # Always close the session
```

The `@contextmanager` pattern ensures every database operation is wrapped in a proper transaction. This eliminates manual commit/rollback management throughout the codebase — callers simply use `with self.get_session() as session:` and the context manager handles all lifecycle concerns.

Python Reference: contextlib.contextmanager — how the `@contextmanager` decorator turns a generator into a context manager.

4.2 Authentication System

Password hashing uses Werkzeug's `generate_password_hash()` and `check_password_hash()` which implement PBKDF2-SHA256 with random salts. Two separate authentication methods serve different user types:

```
# Staff authentication (Qt desktop) — returns SQLAlchemy User object
def authenticate_user(self, username: str, password: str) -> User | None:
```

```
    with self.get_session() as session:
        user = session.query(User).filter_by(username=username,
is_active=True).first()
        if user and self.verify_password(user.password_hash, password):
            session.expunge(user)   # Detach from session so it survives
session.close()
            return user
    return None

# Patient portal authentication (Flask web) — returns dict (avoids detached-instance
issues)
def authenticate_portal(self, username: str, password: str) -> dict | None:
    with self.get_session() as session:
        account = session.query(PatientPortalAccount).filter_by(
            username=username, is_active=True).first()
        if account and self.verify_password(account.password_hash, password):
            account.last_login = datetime.utcnow()
            result = {"id": account.id, "patient_id": account.patient_id,
                      "username": account.username, "email": account.email}
            session.commit()
            return result           # Dict survives session close without detached
errors
    return None
```

Security Note: The portal method returns a dict instead of an ORM object. This was an intentional design decision after encountering `DetachedInstanceError` when Flask routes tried to access lazy-loaded relationships on expunged objects. Returning dicts creates a clean serialization boundary.

Werkzeug Security: Password Hashing Reference

4.3 Drug Interaction Checking Algorithm

The interaction checker takes a list of medication IDs and performs pairwise lookups against the MedicationInteraction table. Since interactions are stored as unordered pairs (A↔B, not A→B), the query checks both orderings:

```
def check_interactions(self, medication_ids: list[int]) -> list[dict]:
    with self.get_session() as session:
        results = []
        for i, mid_a in enumerate(medication_ids):
            for mid_b in medication_ids[i + 1:]:        # Only check each pair once
                inter = session.query(MedicationInteraction).filter(
                    or_(
                        and_(MedicationInteraction.medication_a_id == mid_a,
                             MedicationInteraction.medication_b_id == mid_b),
                        and_(MedicationInteraction.medication_a_id == mid_b,
                             MedicationInteraction.medication_b_id == mid_a)
                    )
                ).first()
                if inter:
                    results.append({
                        "medication_a": med_a.brand_name,
                        "medication_b": med_b.brand_name,
                        "severity": inter.severity.value,   #
"minor"|"moderate"|"major"|"contraindicated"
                        "description": inter.description     # Clinical interaction
description
```

```
                })
        return results
```

This method is called by the prescription creation dialog in the Qt app (via the "Check Interactions" button) and returns interaction data with severity levels. The UI displays MAJOR and CONTRAINDICATED interactions in red to alert prescribers.

4.4 Complete API Method Reference

Method	Signature	Description
authenticate_user	(username, password) → User|None	Staff login via username/password
authenticate_portal	(username, password) → dict|None	Patient portal login
create_patient	(**kwargs) → int	Create patient, returns ID
get_patient_full	(patient_id) → dict|None	Full patient with allergies, insurance, diagnoses
search_patients	(query) → list[dict]	Search by name/email/phone (ILIKE)
search_medications	(query, class, schedule, form) → list	Multi-filter medication search
check_interactions	(medication_ids) → list[dict]	Pairwise drug interaction check
create_prescription	(patient, prescriber, items, ...) → int	Create Rx with line items + pricing
create_invoice_from_prescription	(rx_id) → int|None	Auto-generate invoice from Rx items
record_payment	(invoice_id, amount, method, ...) → int	Process payment, update balance
get_dashboard_stats	() → dict	KPIs: patients, Rx, revenue, appointments
get_revenue_by_month	(months) → list[dict]	Monthly revenue for analytics charts
get_top_medications	(limit) → list[dict]	Most-prescribed medications ranked
log_action	(user_id, action, entity, ...) → None	HIPAA audit trail logging

Table 4.1: DatabaseManager method reference (15 of 40+ methods shown)

5. Seed Data — Medications, Symptoms & Conditions

The `database/seed_data.py` file populates the database with realistic clinical data including 55 real-world medications with actual NDC codes, FDA indications, contraindications, side effects, and market pricing. The seed data also establishes 24 clinically-accurate drug-drug interactions with severity levels and descriptions.

5.1 Medication Categories

Category	Count	Examples	DEA Schedule
Antibiotics	5	Amoxicillin, Azithromycin, Ciprofloxacin, Doxycycline, Cephalexin	None
Antidepressants (SSRI/SNRI)	5	Zoloft, Prozac, Lexapro, Effexor XR, Wellbutrin XL	None
Antipsychotics (Atypical)	3	Zyprexa, Risperdal, Seroquel	None
Benzodiazepines	3	Xanax, Klonopin, Ativan	Schedule IV
CNS Stimulants (ADHD)	3	Adderall XR, Ritalin, Vyvanse	Schedule II
Antihypertensives	5	Lisinopril, Losartan, Amlodipine, Metoprolol, HCTZ	None
Statins	2	Lipitor, Crestor	None
Antidiabetics	3	Metformin (Glucophage), Januvia, Jardiance	None
Anticoagulants	3	Eliquis, Xarelto, Warfarin	None
Pain / Neuropathic	5	Tylenol #3, Tramadol, Naproxen, Gabapentin, Pregabalin	III/IV/V/None
Respiratory	2	Ventolin HFA, Symbicort	None
Other Specialties	16	Synthroid, Prednisone, Ambien, Viagra, Imitrex, etc.	Various

Table 5.1: Medication categories in the seed database

5.2 Drug Interaction Examples

24 clinically-significant interactions are pre-loaded, representing real pharmacological concerns that prescribers must evaluate. Severity levels follow standard clinical classifications:

Drug A	Drug B	Severity	Clinical Concern
Warfarin	Naproxen	MAJOR	Greatly increased bleeding risk; NSAIDs impair platelet function

Drug A	Drug B	Severity	Clinical Concern
Xanax	Ambien	MAJOR	Combined CNS depression: profound sedation, respiratory failure risk
Zoloft	Tramadol	MAJOR	Serotonin syndrome risk: agitation, hyperthermia, tachycardia
Lithium	Lisinopril	MAJOR	ACE inhibitors reduce lithium clearance → toxicity risk
Depakote	Lamictal	MAJOR	Valproate doubles lamotrigine levels → Stevens-Johnson syndrome
Lipitor	Diflucan	MAJOR	CYP3A4 inhibition → significantly increased statin levels → rhabdomyolysis
Prozac	Zyprexa	MODERATE	CYP2D6 inhibition increases olanzapine levels and sedation
Viagra	Amlodipine	MODERATE	Additive hypotension: symptomatic blood pressure drops
Synthroid	Prilosec	MODERATE	PPIs reduce levothyroxine absorption; requires dose monitoring

Table 5.2: Representative drug interactions (9 of 24 shown)

Clinical Reference: Drug interaction data sourced from pharmacological literature. See Drugs.com Interaction Checker and Medscape Drug Reference for comprehensive databases.

5.3 Seed Data Population Pattern

```
def seed_database(db_manager: DatabaseManager):
    if not db_manager.is_database_empty():      # Idempotent — only seeds empty
databases
        return

    with db_manager.get_session() as session:
        # Phase 1: Create staff users (5 users with hashed passwords)
        users = [User(username="dr.carter",
password_hash=generate_password_hash("doctor123"),
                      role=UserRole.DOCTOR, ...), ...]
        session.add_all(users); session.flush()

        # Phase 2: Create patients (15 patients with full demographics)
        # Phase 3: Create portal accounts (5 patients get web access)
        # Phase 4: Create insurance records (10 policies across providers)

        # Phase 5: Create 55 medications with real FDA data
        meds_data = [
            (ndc_code, brand_name, generic_name, manufacturer, drug_class,
             schedule, route, form, strength, unit, description,
             indications, contraindications, side_effects, awp, retail_price),
            ...
        ]

        # Phase 6: Create 24 drug-drug interactions
        # Phase 7: Patient clinical data (allergies, vitals, diagnoses, records)
        # Phase 8: Prescriptions with line items
        # Phase 9: Invoices, payments, appointments
        # Phase 10: Audit log entries
```

6. Qt Desktop Application — Architecture

The desktop application is built with PyQt6, Qt's Python binding for cross-platform GUI development. The application follows a single-window design with a persistent sidebar navigation and a stacked widget container that swaps between 8 functional modules.

PyQt6 Documentation: PyQt6 Reference Guide | Qt 6 C++ Reference (concepts map directly to PyQt6)

6.1 Application Lifecycle

```
# run_qt.py — Entry point
def main():
    db_manager = DatabaseManager("data/medpharm.db")
    db_manager.init_db()
    seed_database(db_manager)                    # Idempotent seeding

    app = QApplication(sys.argv)
    window = MainWindow(db_manager)              # Triggers login dialog
    window.showMaximized()
    sys.exit(app.exec())

# MainWindow.__init__() lifecycle:
# 1. Set stylesheet (400+ line dark theme from styles.py)
# 2. Show LoginDialog (modal) — blocks until authentication
# 3. Build sidebar with 8 navigation buttons
# 4. Create QStackedWidget with all 8 module widgets
# 5. Pass db_manager + current_user dict to each widget
# 6. Navigate to Dashboard (index 0)
```

6.2 Navigation System

The sidebar uses `QPushButton` widgets with `setCheckable(True)` to create radio-button behavior. When a nav button is clicked, `navigate_to(index)` updates the `QStackedWidget` and calls `refresh_data()` on the target widget:

```
NAV_ITEMS = [
    ("Dashboard", "dashboard"),      # Index 0 — DashboardWidget
    ("Patients", "patients"),         # Index 1 — PatientWidget
    ("Prescriptions", "prescriptions"), # Index 2 — PrescriptionWidget
    ("Medications", "medications"),   # Index 3 — MedicationWidget
    ("Appointments", "appointments"), # Index 4 — AppointmentWidget
    ("Records", "records"),           # Index 5 — RecordsWidget
    ("Billing", "billing"),           # Index 6 — BillingWidget
    ("Analytics", "analytics"),       # Index 7 — AnalyticsWidget
]

def navigate_to(self, index: int):
    for i, btn in enumerate(self.nav_buttons):
        btn.setChecked(i == index)          # Visual: highlight active nav
    self.stack.setCurrentIndex(index)        # Swap visible widget
    widget = self.stack.currentWidget()
    if hasattr(widget, "refresh_data"):
        widget.refresh_data()               # Reload data from database
```

6.3 Stylesheet Architecture

The `qt_app/styles.py` file contains a 400+ line Qt stylesheet (QSS) that implements a professional dark medical theme with teal (#00BCD4) accents. QSS is syntactically similar to CSS but targets Qt widget classes. Key patterns used:

```
# Object name selectors for role-specific styling
QPushButton#primary_button { background-color: #00BCD4; color: #ffffff; border: none;
}
QPushButton#danger_button  { background-color: #E53935; color: #ffffff; }
QFrame#kpi_card            { background-color: #1e2129; border-radius: 12px; }

# Status label variants (applied dynamically based on data)
QLabel#status_active  { background: rgba(67,160,71,0.2); color: #43A047;
border-radius: 10px; }
QLabel#status_pending { background: rgba(251,140,0,0.2); color: #FB8C00;
border-radius: 10px; }

# Table styling with alternating rows and teal headers
QHeaderView::section { background: #1e2129; color: #00BCD4; border-bottom: 2px solid
#00BCD4; }
QTableWidget { alternate-background-color: #161920; selection-background-color:
rgba(0,188,212,0.2); }
```

Qt Stylesheet Reference: Qt 6 Stylesheet Syntax | Property Reference

7. Qt Desktop — Dashboard & Navigation

The `DashboardWidget` serves as the landing page after login, providing a real-time overview of the practice with KPI cards, today's appointments, and recent activity. It uses a `QTimer` for automatic 60-second refresh cycles.

7.1 KPI Card Construction

```
def _create_kpi_card(self, label, value, color):
    frame = QFrame()
    frame.setObjectName("kpi_card")
    frame.setStyleSheet(f"""
        QFrame#kpi_card {{ border-top: 3px solid {color}; }}
    """)                                          # Dynamic color per card
    layout = QVBoxLayout(frame)

    val_label = QLabel(value)
    val_label.setObjectName("card_value")           # Styled: 28px bold white
    val_label.setAlignment(Qt.AlignmentFlag.AlignCenter)
    layout.addWidget(val_label)

    name_label = QLabel(label)
    name_label.setObjectName("card_label")          # Styled: 12px gray
    layout.addWidget(name_label)

    return {"frame": frame, "value_label": val_label}

# Called with 6 KPIs:
#   ("patient_count",        "Total Patients",    "#00BCD4")
#   ("active_prescriptions", "Active Rx",         "#43A047")
#   ("today_appointments",   "Today's Appts",     "#FB8C00")
#   ("monthly_revenue",      "Monthly Revenue",   "#7C4DFF")
#   ("pending_bills",        "Pending Bills",     "#E53935")
#   ("controlled_substances","Controlled Rx",     "#FF5722")
```

8. Qt Desktop — Patient Management

The `PatientWidget` implements a master-detail layout using `QSplitter`. The left panel provides patient search and listing; the right panel displays a 7-tab detail view (Demographics, Insurance, Allergies, Vitals, Diagnoses, Prescriptions, Billing).

8.1 Real-Time Search

```
# QLineEdit.textChanged signal triggers search on every keystroke
self.search_input.textChanged.connect(self.search_patients)

def search_patients(self, query=""):
    patients = self.db_manager.search_patients(query)   # ILIKE on name, email, phone
    self.patient_table.setRowCount(len(patients))
    for i, p in enumerate(patients):
        self.patient_table.setItem(i, 0, QTableWidgetItem(str(p["id"])))
        self.patient_table.setItem(i, 1, QTableWidgetItem(p["full_name"]))
        # ... populate remaining columns
```

8.2 Patient Detail Tabs

When a patient row is selected, `currentCellChanged` fires `on_patient_selected()` which calls `load_patient_details(pid)`. This method populates all 7 tabs in a single pass by calling multiple DatabaseManager methods:

Tab	Data Source	Content
Demographics	get_patient_full()	Name, DOB/age, gender, address, emergency contact, blood type
Insurance	get_patient_full().insurance_records	Provider, policy#, group#, copay, coverage type
Allergies	get_patient_full().allergies	Allergen, reaction description, severity (color-coded)
Vitals	get_patient_vitals()	BP, HR, temp, RR, SpO2, weight, BMI (time-series table)
Diagnoses	get_patient_full().diagnoses	ICD-10 code, description, status, date, provider
Prescriptions	get_prescriptions_by_patient()	Rx#, medications, status, date, prescriber
Billing	get_patient_invoices()	Invoice#, date, total, paid, balance, status

Table 8.1: Patient detail tab data sources

9. Qt Desktop — Prescription & Drug Interaction Checking

The prescription creation dialog is the most complex UI component, combining patient search, dynamic medication row addition, drug interaction checking, and invoice generation.

9.1 Prescription Dialog Workflow

Step	UI Component	Action
1. Select Patient	QLineEdit + QComboBox	Type-ahead search populates combo with matching patients
2. Add Medications	Dynamic QFrame rows	Each row: medication search, dosage, frequency, qty, refills, instructions
3. Check Interactions	QPushButton (orange)	Calls check_interactions() with selected med IDs; shows results in QLabel
4. Review Warnings	QLabel (red/green)	MAJOR/CONTRAINDICATED shown in red; safe combinations in green
5. Create Rx	QPushButton (teal)	Calls create_prescription() → generates Rx with items + pricing
6. Generate Invoice	Detail dialog button	Calls create_invoice_from_prescription() for billing

10. Qt Desktop — Medication Database

The `MedicationWidget` provides a searchable, filterable view of all 55 medications with a detail panel showing complete drug information. Features include multi-filter search (by name, drug class, DEA schedule), CSV export, and simulated market price updates.

10.1 Detail Panel Information

When a medication row is selected, the right panel displays grouped information sections: drug information (manufacturer, class, schedule, route, form, strength, NDC), pricing (AWP and retail with last update timestamp), indications/uses, contraindications, side effects, and known drug interactions with severity-coded display (red for MAJOR, orange for MODERATE).

11. Qt Desktop — Billing, Insurance, Symptoms, Records, Analytics

Module	Key Feature	Widget Classes Used
Appointments	Calendar view + status workflow (6 status transitions)	QCalendarWidget, QSplitter, QDateTimeEdit, QSpinBox

Module	Key Feature	Widget Classes Used
Medical Records	Patient selection + type filtering + content viewer	QComboBox, QSplitter, QTextEdit (read-only)
Billing	Revenue summary KPIs + payment recording + invoice detail dialog	QDoubleSpinBox, QFrame#kpi_card, QDialog, QScrollArea
Analytics	4 matplotlib charts: revenue, demographics, top meds, provider workload	FigureCanvasQTAgg, Figure, Axes (matplotlib)

Table 11.1: Remaining Qt module summary

matplotlib Qt Integration: Embedding matplotlib in Qt — the `FigureCanvasQTAgg` backend renders matplotlib figures directly in Qt widgets. Chart styling uses dark backgrounds (`facecolor="#1e2129"`) and teal/orange palette to match the UI theme.

12. Flask Web Portal — Application Factory & Routing

The patient-facing web portal is built with Flask using the application factory pattern. The `create_app(db_manager)` function configures the Flask instance, registers the portal blueprint, sets up request hooks for user loading, and configures error handlers.

Flask Documentation: Application Factory Pattern | Blueprints and Views

12.1 Application Factory

```
def create_app(db_manager):
    app = Flask(__name__,
                template_folder="templates",     # Jinja2 template directory
                static_folder="static")          # CSS/JS/images

    app.secret_key = secrets.token_hex(32)       # Session encryption key
    app.config["DB_MANAGER"] = db_manager        # Store db_manager in app config

    @app.before_request
    def load_user():
        g.db_manager = app.config["DB_MANAGER"]  # Make db_manager available via
Flask g
        g.user_id = session.get("user_id")        # Load session data into request
globals
        g.patient_id = session.get("patient_id")

    @app.context_processor
    def inject_globals():
        return {                                  # Variables available in ALL
templates
            "current_year": 2026,
            "user_logged_in": "user_id" in session,
            "patient_name": session.get("patient_name", ""),
        }

    app.register_blueprint(portal_bp)            # Mount all routes from routes.py
    return app
```

12.2 Route Architecture

Route	Methods	Auth	Description
/login	GET, POST	No	Portal login form + authentication
/logout	GET	No	Clear session and redirect to login
/register	GET, POST	No	Patient identity verification + account creation

Route	Methods	Auth	Description
/dashboard	GET	Yes	KPIs, active Rx, upcoming appointments, recent invoices
/prescriptions	GET	Yes	Tabbed list: active, all, past prescriptions
/prescriptions/	GET	Yes	Full prescription detail with medication info
/prescriptions//refill	POST	Yes	Submit refill request for eligible items
/billing	GET	Yes	Invoice list + payment history
/billing//pay	GET, POST	Yes	Payment form + processing
/records	GET	Yes	Medical records with type filtering
/appointments	GET	Yes	Upcoming + past appointments
/medications	GET	Yes	Current active medications with drug info
/profile	GET, POST	Yes	View/edit contact info, insurance, allergies
/api/notifications	GET	Yes	JSON: overdue bills, upcoming appointments
/api/prescriptions	GET	Yes	JSON: all patient prescriptions
/api/invoices/outstanding	GET	Yes	JSON: unpaid invoices

Table 12.1: Complete Flask route reference

12.3 Authentication Decorator

```
def login_required(f):
    @wraps(f)                                    # Preserve function metadata
    def wrapper(*args, **kwargs):
        if "user_id" not in session:             # Check Flask session for auth
            flash("Please log in to continue.", "warning")
            return redirect(url_for("portal.login"))
        return f(*args, **kwargs)
    return wrapper

# Usage: every protected route gets @login_required
@portal_bp.route("/dashboard")
@login_required
def dashboard():
    patient_id = session["patient_id"]           # Safe: login_required guarantees
session
    prescriptions = g.db_manager.get_prescriptions_by_patient(patient_id)
    ...  # Only this patient's data is ever queried
```

13. Flask Web Portal — Templates & Frontend

The portal uses 14 Jinja2 templates with a shared base layout. The frontend leverages Bootstrap 5's dark theme with a custom 450-line CSS override for the medical color palette. Font Awesome 6.5 provides the icon system. JavaScript handles notifications, card formatting, and dynamic form behavior.

13.1 Template Inheritance

```
{# base.html — Master layout #}
<html data-bs-theme="dark">                        {# Bootstrap 5 dark mode #}
<head>
    <link href="bootstrap@5.3.2/dist/css/bootstrap.min.css" rel="stylesheet">
    <link href="font-awesome/6.5.1/css/all.min.css" rel="stylesheet">
    <link href="{{ url_for('static', filename='css/style.css') }}" rel="stylesheet">
</head>
<body>
    {% if user_logged_in %}
    <nav class="navbar">                           {# Shown only when authenticated #}
        ... 7 navigation links + notification bell + user dropdown ...
    </nav>
    {% endif %}

    <main>
        {% with messages = get_flashed_messages(with_categories=true) %}
            {% for category, message in messages %}
            <div class="alert alert-{{ category }} alert-dismissible">
                {{ message }}                      {# Flash messages with auto-dismiss
#}
            </div>
            {% endfor %}
        {% endwith %}

        {% block content %}{% endblock %}          {# Child template content #}
    </main>
</body>
```

Jinja2 Reference: Template Designer Documentation — covers template inheritance, macros, filters, and control structures used throughout.

13.2 Key Jinja2 Pattern: Dict Key Access

An important implementation detail: when iterating prescription dicts, the key `items` conflicts with Python dict's `.items()` method. Jinja2 resolves `rx.items` as the method call, not the dict key. The fix uses bracket notation:

```
{# WRONG — rx.items resolves to dict.items() method, not the "items" key #}
{% for item in rx.items %}         {# TypeError: 'builtin_function_or_method' not
iterable #}

{# CORRECT — bracket notation accesses the dict key #}
```

```
{% for item in rx["items"] %}      {# Iterates the prescription's medication items
list #}
    {{ item.medication_name }} — {{ item.dosage }}
{% endfor %}
```

Gotcha: This is a common Jinja2 pitfall when dict keys shadow Python built-in method names (items, keys, values, get, pop, update). Always use bracket notation for dict keys that match built-in names.

14. Flask Web Portal — Patient Self-Service Features

14.1 Online Bill Pay Flow

Step	Route/Template	Implementation Detail
1. View invoices	/billing → billing.html	get_patient_invoices() filtered by patient_id; outstanding balance calculated
2. Click Pay	/billing//pay GET → pay.html	Invoice detail + payment form rendered; max amount = balance_due
3. Submit payment	/billing//pay POST	record_payment() updates invoice balance; status transitions to PARTIAL or PAID
4. Confirmation	Flash message → /billing redirect	Success flash with amount; invoice table refreshes with new status

14.2 Prescription Refill Request

```
@portal_bp.route("/prescriptions/<int:rx_id>/refill", methods=["POST"])
@login_required
def request_refill(rx_id):
    rx = g.db_manager.get_prescription(rx_id)
    # Security: verify this prescription belongs to the logged-in patient
    if not rx or rx["patient_id"] != session["patient_id"]:
        flash("Prescription not found.", "danger")
        return redirect(url_for("portal.prescriptions"))

    refilled = False
    for item in rx.get("items", []):
        if item["refills_remaining"] > 0:
            g.db_manager.refill_prescription_item(item["id"])  # Increments
refills_used
            refilled = True

    if refilled:
        flash("Refill request submitted successfully.", "success")
    else:
        flash("No refills remaining for this prescription.", "warning")
    return redirect(url_for("portal.prescription_detail", rx_id=rx_id))
```

14.3 Patient Registration with Identity Verification

Patient registration requires identity verification against existing records in the system. The patient must provide their first name, last name, date of birth, and last 4 digits of their SSN. These are matched against the Patient table to prevent unauthorized account creation:

```
def verify_patient_identity(self, first_name, last_name, dob, ssn_last4) -> int |
None:
    with self.get_session() as session:
        patient = session.query(Patient).filter(
            func.lower(Patient.first_name) == first_name.lower(),  #
Case-insensitive
            func.lower(Patient.last_name) == last_name.lower(),
            Patient.dob == dob,                                     # Exact DOB match
            Patient.ssn_last4 == ssn_last4,                         # SSN
verification
            Patient.is_active == True
        ).first()
        if patient and not patient.portal_account:                  # No duplicate
accounts
            return patient.id
    return None
```

15. Cloud REST API — Authentication & Endpoints

The Cloud REST API provides JSON endpoints for all mobile and desktop clients. It uses HMAC-SHA256 JWT tokens for authentication, with separate login flows for patients and staff. The API is CORS-enabled and can be deployed via Gunicorn or Docker.

15.1 Authentication Endpoints

Method	Endpoint	Description
POST	/api/v1/auth/login/patient	Patient login, returns JWT tokens
POST	/api/v1/auth/login/staff	Staff login, returns JWT tokens
POST	/api/v1/auth/register	Patient registration with identity verification
POST	/api/v1/auth/refresh	Refresh access token using refresh token

Table 15.1: Authentication endpoints

15.2 Patient API Endpoints

Method	Endpoint	Description
GET	/api/v1/patient/dashboard	Dashboard KPIs and summaries
GET	/api/v1/patient/prescriptions	List prescriptions (filterable by status)
POST	/api/v1/patient/prescriptions/:id/refill	Request prescription refill
GET	/api/v1/patient/billing	Invoices, payments, outstanding balance
POST	/api/v1/patient/billing/:id/pay	Submit payment for invoice
GET	/api/v1/patient/appointments	List appointments
GET	/api/v1/patient/records	Medical records (filterable by type)
GET/PUT	/api/v1/patient/profile	View/update patient profile
GET	/api/v1/patient/insurance	List insurance records
GET/POST	/api/v1/patient/insurance/claims	List/submit insurance claims
GET	/api/v1/patient/notifications	System notifications

Table 15.2: Patient API endpoints

15.3 Reference Data Endpoints

Reference data endpoints require any valid token (patient or staff):

Method	Endpoint	Description
GET	/api/v1/medications/search	Search medications by name, class, schedule
GET	/api/v1/reference/symptoms	Search symptoms by name, body system
GET	/api/v1/reference/conditions	Search conditions by name, ICD-10, category
GET	/api/v1/reference/symptoms/body-systems	List all body systems
GET	/api/v1/reference/conditions/categories	List all condition categories

Table 15.3: Reference data endpoints

16. Mobile & Desktop Clients

16.1 Android (Kotlin)

The Android client uses MVVM architecture with Retrofit for networking, OkHttp interceptors for automatic JWT token injection, and EncryptedSharedPreferences for secure token storage. The UI uses Material 3 components with bottom navigation (Dashboard, Prescriptions, Billing, Appointments, Records) and a toolbar menu for Profile, Medications, and Logout.

Component	Technology	Purpose
Networking	Retrofit 2 + OkHttp	Type-safe REST API calls with auth interceptor
Architecture	ViewModel + LiveData	MVVM pattern with lifecycle awareness
Data	Repository pattern	Single source of truth for API data
Security	EncryptedSharedPreferences	AES-256 encrypted token storage
UI	Material 3 + ViewBinding	Dark theme, swipe-to-refresh, RecyclerView

Table 16.1: Android technology stack

16.2 iOS (SwiftUI)

The iOS client uses native SwiftUI with async/await networking via an actor-based APIClient. Authentication tokens are stored in iOS Keychain Services. The app uses NavigationStack with tab navigation: Dashboard, Prescriptions, Billing, Appointments, and a More section containing Records, Medications, Symptoms, Profile, and Notifications.

16.3 macOS (SwiftUI)

The macOS client shares Models and APIClient code with iOS. It uses NavigationSplitView for a native sidebar layout with master-detail navigation. SwiftUI Table components provide sortable data grids for prescriptions and medications. The app supports all features including insurance claims, medical records, and profile management.

16.4 Windows (.NET 8 / WPF)

The Windows desktop client uses .NET 8 with WPF and CommunityToolkit.Mvvm. It features a sidebar navigation layout with pages for Dashboard, Prescriptions, Billing, Appointments, and Insurance Claims. Authentication tokens are encrypted using Windows DPAPI (Data Protection API) and stored at %APPDATA%/MedPharm/session.dat.

17. Insurance Claims Workflow

Insurance claims follow a state machine workflow from submission through processing:

State	Trigger	Next State	Actions
SUBMITTED	Patient files claim	IN_REVIEW	Claim record created, claim number generated
IN_REVIEW	Staff reviews claim	APPROVED/DENIED	Staff sets approved amount or denial reason
APPROVED	Staff processes payment	PAID	Payment auto-created on invoice, balance updated
DENIED	Final state	—	Denial reason recorded, patient notified
PAID	Final state	—	Insurance payment recorded, invoice balance reduced

Table 17.1: Insurance claim state transitions

Claims are available across all platforms: Qt desktop (staff submission and processing), Android, iOS, macOS, and Windows (patient submission via REST API). Staff can process claims through the Qt desktop billing module or via the staff API endpoint.

18. Authentication & Security Model

Layer	Mechanism	Implementation
Password Storage	PBKDF2-SHA256 with random salts	Werkzeug generate_password_hash()
Session (Web)	Server-side encrypted cookies	Flask session with 256-bit secret key
JWT Auth (API)	HMAC-SHA256 signed tokens	Access + refresh tokens with configurable expiry
Access Control	Role-based (4 roles)	UserRole enum: doctor, psychiatrist, pharmacist, admin
Patient Isolation	Session/token-scoped patient_id	All queries filter by patient_id from session/JWT
Token Storage	Platform-specific encryption	Android: EncryptedSharedPrefs, iOS: Keychain, Win: DPAPI
Identity Verification	4-factor registration	First name + last name + DOB + SSN last 4
SQL Injection	Parameterized queries	SQLAlchemy ORM handles all query parameterization
XSS Prevention	Auto-escaping templates	Jinja2 autoescapes all {{ }} expressions
CSRF Protection	Flask session tokens	Session-based state prevents cross-site requests
Audit Logging	Action logging with user/timestamp	AuditLog table tracks all data changes

Table 18.1: Security control matrix

OWASP Reference: OWASP Top 10 Web Security Risks — the architecture addresses injection (A03), authentication (A07), and access control (A01).

HIPAA Compliance Note: This system is designed with HIPAA-aware patterns (audit logging, role-based access, patient data isolation) but a production deployment would require additional controls: encrypted data at rest, TLS in transit, BAA agreements, access reviews, and formal risk assessment.

19. Deployment & Configuration Guide

19.1 Automated Installation

```
# One-command installation
cd MedPharm
chmod +x install.sh
./install.sh

# The installer:
#   1. Detects OS (Ubuntu/Debian, Fedora, macOS, Arch)
#   2. Checks Python 3.10+ availability
#   3. Creates virtual environment
#   4. Installs all dependencies (Flask, SQLAlchemy, PyQt6, matplotlib, reportlab)
#   5. Initializes SQLite database with seed data
#   6. Creates launcher scripts (start_web.sh, start_desktop.sh, generate_docs.sh)
#   7. Runs integration tests to validate installation
```

19.2 Manual Setup

```
# 1. Create virtual environment
python3 -m venv venv && source venv/bin/activate

# 2. Install dependencies
pip install Flask SQLAlchemy Werkzeug matplotlib numpy PyQt6 reportlab

# 3. Launch web portal (patients)
python3 run_web.py              # → http://localhost:5000

# 4. Launch desktop app (staff) — requires display server
python3 run_qt.py               # Opens Qt window with login dialog

# 5. Generate PDF documentation
python3 docs/generate_pdf.py    # → docs/MedPharm_ERP_Documentation.pdf
```

19.3 Configuration

Setting	Location	Default	Description
Database path	run_qt.py / run_web.py	data/medpharm.db	SQLite database file path
Web port	run_web.py CLI arg	5000	Flask development server port
Debug mode	run_web.py	True	Flask debug mode (disable in production)
Session secret	web/app.py	Random 256-bit	Generated per-instance; set fixed for production
Auto-refresh	DashboardWidget	60 seconds	QTimer interval for dashboard KPI updates

19.4 Production Deployment Recommendations

- Replace SQLite with PostgreSQL for concurrent multi-user access
- Deploy Flask behind Gunicorn + Nginx with TLS certificates
- Set a fixed SECRET_KEY via environment variable (not random per-restart)
- Enable CSRF token validation on all POST forms
- Implement rate limiting on /login and /api endpoints
- Add database encryption at rest for HIPAA compliance
- Configure log rotation for audit_log table
- See Flask Deployment Options for complete guidance

20. Docker Server Deployment (Ubuntu)

MedPharm ERP includes a production-ready Docker server package based on Ubuntu Server 24.04 LTS. Two deployment options are provided: a lightweight API-only container and a full-stack server with Nginx reverse proxy. Both images are published to Docker Hub under the enlightec namespace and can be pulled directly without a local build.

20.1 Deployment Modes

MedPharm ships four docker compose files — two that build from source and two that pull the pre-built images from Docker Hub:

File	Image	Source
docker-compose.yml	medpharm-api (local build)	Dockerfile
docker-compose.hub.yml	enlightec/medpharm-api:latest	Docker Hub
server/docker-compose.yml	medpharm-server (local build)	server/Dockerfile
server/docker-compose.hub.yml	enlightec/medpharm-server:latest	Docker Hub

Table 20.0: Docker Compose files

20.2 Full Server Stack (Recommended)

The full stack runs three services managed by Supervisor inside a single container: Nginx reverse proxy (port 80), Cloud REST API via Gunicorn (port 8080), and Patient Web Portal via Gunicorn (port 5000).

```
# Deploy the full server stack
cd server
cp .env.example .env           # Configure secrets and ports
docker compose up -d           # Start all services
docker compose logs -f         # View logs
docker compose down            # Stop

# Endpoints via Nginx (port 80):
#   http://localhost/api/v1/health    REST API health check
#   http://localhost/portal/          Patient Web Portal
#   http://localhost:8080/            API direct access
#   http://localhost:5000/            Web Portal direct access
```

20.3 API-Only Container

For deployments that only need the REST API (e.g., serving mobile clients), the root-level Dockerfile provides a lightweight Ubuntu container running Gunicorn without Nginx or the web portal.

```
# API-only deployment
docker compose up -d           # Starts API on port 8080
```

```
curl http://localhost:8080/api/v1/health
```

20.4 Zero-Build Deployment via Docker Hub

The pre-built images published to Docker Hub let operators deploy MedPharm ERP without cloning the source or running any Python install. The install.sh script supports --docker and --docker-server flags that pull the image and start the container in a single step. A dedicated start_docker_hub.sh helper provides an interactive launcher.

```
# One-line installers (pulls from Docker Hub automatically)
./install.sh --docker              # API only on port 8080
./install.sh --docker-server       # Full stack on port 80
./install.sh --docker --tag=1.1.1  # Pin to a specific release

# Interactive launcher
./start_docker_hub.sh              # Menu
./start_docker_hub.sh server       # Full stack
./start_docker_hub.sh api          # API only
./start_docker_hub.sh pull         # Pull latest, do not start
./start_docker_hub.sh stop         # Stop all MedPharm containers

# Or use docker compose directly against the Hub compose files
docker compose -f docker-compose.hub.yml up -d             # API only
cd server && docker compose -f docker-compose.hub.yml up -d # Full stack
```

20.5 Docker Architecture

Component	Base Image	Port	Process Manager
Full Stack	Ubuntu 24.04 LTS	80, 8080, 5000	Supervisor + Nginx
API Only	Ubuntu 24.04 LTS	8080	Gunicorn

20.6 Environment Variables

Variable	Default	Description
MEDPHARM_JWT_SECRET	dev default	JWT signing secret (change in production)
MEDPHARM_SECRET_KEY	auto-generated	Flask secret key
MEDPHARM_DB_PATH	/data/medpharm_erp.db	SQLite database path
MEDPHARM_WORKERS	4	Gunicorn worker processes
MEDPHARM_THREADS	2	Gunicorn threads per worker
MEDPHARM_HTTP_PORT	80	Nginx listen port (full stack)
MEDPHARM_API_PORT	8080	API server port
MEDPHARM_WEB_PORT	5000	Web portal port
MEDPHARM_CORS_ORIGINS	*	Allowed CORS origins

Variable	Default	Description
MEDPHARM_TOKEN_EXPIRY	86400	Access token lifetime (seconds)
MEDPHARM_REFRESH_EXPIRY	604800	Refresh token lifetime (seconds)

Table 20.1: Docker environment variables

20.7 Nginx Routing

The Nginx reverse proxy routes requests to the appropriate backend service. Security headers (X-Frame-Options, X-Content-Type-Options, X-XSS-Protection) are added to all responses. CORS preflight requests are handled at the proxy level.

Path	Upstream	Description
/api/*	gunicorn :8080	Cloud REST API
/portal/*	gunicorn :5000	Patient Web Portal
/	gunicorn :8080	API root info
/health	gunicorn :8080	Health check (no logging)

20.8 Persistent Storage

The Docker deployment uses named volumes for persistent data. The SQLite database is stored at /data/medpharm_erp.db inside the container, mapped to the medpharm-data volume. Server logs are stored in /var/log/medpharm/, mapped to the medpharm-logs volume.

Security note: Always set `MEDPHARM_JWT_SECRET` and `MEDPHARM_SECRET_KEY` to unique, random values in production. The .env.example file provides a template for all configurable settings.

20.9 CI/CD Pipeline (GitHub Actions)

Docker images are automatically built and pushed to Docker Hub on every tagged release via the GitHub Actions workflow at .github/workflows/docker-publish.yml. The pipeline first validates source code (syntax checks, database init, API health check, web portal login page), then builds and pushes two images to Docker Hub under the enlightec namespace.

```
# Trigger a release build
git tag v1.2.0
git push origin v1.2.0

# Pre-built images on Docker Hub (https://hub.docker.com/u/enlightec):
docker pull enlightec/medpharm-server:latest   # Full stack
docker pull enlightec/medpharm-api:latest      # API only

# Or pin to a specific release
docker pull enlightec/medpharm-server:1.1.1
docker pull enlightec/medpharm-api:1.1.1
```

Image	Docker Hub URL	Contents
enlightec/medpharm-server	hub.docker.com/r/enlightec/medpharm-server	Nginx + API + Web Portal
enlightec/medpharm-api	hub.docker.com/r/enlightec/medpharm-api	REST API only

Table 20.2: Docker Hub images

Required GitHub Secrets: `DOCKERHUB_USERNAME` (e.g., enlightec) and `DOCKERHUB_TOKEN` (Docker Hub access token). Set these in the repository Settings under Secrets and variables > Actions.

21. API Reference

The Flask portal exposes 3 JSON API endpoints for AJAX consumption by the frontend JavaScript. All endpoints require an authenticated session and return data scoped to the logged-in patient only.

21.1 GET /api/notifications

```
# Response: { "count": 3, "notifications": [
#   { "type": "danger",  "message": "Invoice INV-XXXXX is overdue" },
#   { "type": "warning", "message": "Invoice INV-XXXXX has balance $150.00" },
#   { "type": "info",    "message": "Upcoming appointment: 2026-03-28T09:00" }
# ]}
#
# Polled every 30 seconds by app.js for notification badge updates
```

21.2 GET /api/prescriptions

```
# Response: [
#   { "id": 1, "rx_number": "RX-00000001", "status": "active",
#     "patient_name": "John Smith", "prescriber_name": "Dr. Carter",
#     "prescribed_date": "2026-02-15", "expiry_date": "2026-12-12",
#     "items": [{ "medication_name": "Lisinopril", "dosage": "10mg",
#                 "frequency": "Once daily", "refills_remaining": 2 }],
#     "total": 8.99 },
#   ...
# ]
```

21.3 GET /api/invoices/outstanding

```
# Response: [
#   { "id": 5, "invoice_number": "INV-00000005", "status": "overdue",
#     "total_amount": 339.99, "amount_paid": 0.0, "balance_due": 339.99,
#     "due_date": "2026-03-15",
#     "items": [{ "description": "Psychiatric Evaluation - ADHD", ... }] }
# ]
```

22. Appendix — Technology References

22.1 Essential Documentation Links

- **Python 3 Standard Library**: https://docs.python.org/3/library/index.html
- **SQLAlchemy 2.0 ORM Tutorial**: https://docs.sqlalchemy.org/en/20/tutorial/index.html
- **SQLAlchemy Relationship Patterns**: https://docs.sqlalchemy.org/en/20/orm/relationships.html
- **Flask Quickstart Guide**: https://flask.palletsprojects.com/en/3.0.x/quickstart/
- **Flask Session Interface**: https://flask.palletsprojects.com/en/3.0.x/api/#flask.session
- **Jinja2 Template Language**: https://jinja.palletsprojects.com/en/3.1.x/templates/
- **PyQt6 Signals and Slots**: https://www.riverbankcomputing.com/static/Docs/PyQt6/signals_slots.html
- **Qt 6 Stylesheet Reference**: https://doc.qt.io/qt-6/stylesheet-reference.html
- **Qt 6 Widget Gallery**: https://doc.qt.io/qt-6/gallery.html
- **Bootstrap 5 Components**: https://getbootstrap.com/docs/5.3/components/
- **matplotlib Figure API**: https://matplotlib.org/stable/api/figure_api.html
- **matplotlib Qt Backend**: https://matplotlib.org/stable/gallery/user_interfaces/embedding_in_qt_sgskip.html
- **Werkzeug Security Utilities**: https://werkzeug.palletsprojects.com/en/3.0.x/utils/
- **ReportLab User Guide**: https://docs.reportlab.com/reportlab/userguide/ch1_intro/
- **OWASP Top 10 (2021)**: https://owasp.org/www-project-top-ten/
- **HIPAA Security Rule**: https://www.hhs.gov/hipaa/for-professionals/security/index.html
- **FDA NDC Directory**: https://www.fda.gov/drugs/drug-approvals-and-databases/national-drug-code-directory
- **ICD-10 Code Search**: https://www.icd10data.com/
- **Drugs.com Interaction Checker**: https://www.drugs.com/drug_interactions.html
- **Medscape Drug Reference**: https://reference.medscape.com/drugs
- **Docker Documentation**: https://docs.docker.com/
- **Nginx Reverse Proxy Guide**: https://docs.nginx.com/nginx/admin-guide/web-server/reverse-proxy/
- **Supervisor Configuration**: http://supervisord.org/configuration.html
- **Gunicorn Deployment**: https://docs.gunicorn.org/en/stable/deploy.html

22.2 Default Credentials Reference

Application	Username	Password	Role
Desktop (Qt)	dr.carter	doctor123	Doctor — Internal Medicine
Desktop (Qt)	dr.chen	doctor123	Doctor — Family Medicine
Desktop (Qt)	dr.brooks	doctor123	Psychiatrist
Desktop (Qt)	pharm.davis	pharm123	Pharmacist
Desktop (Qt)	admin	admin123	System Administrator
Web Portal	jsmith_portal	patient123	Patient — John Smith
Web Portal	mjohnson_portal	patient123	Patient — Maria Johnson
Web Portal	ewilliams_portal	patient123	Patient — Emily Williams
Web Portal	sdavis_portal	patient123	Patient — Sarah Davis
Web Portal	lmartinez_portal	patient123	Patient — Linda Martinez

Table 18.1: Default credentials (change in production)

END OF DOCUMENT

This document was auto-generated on April 16, 2026 at 19:30 using ReportLab 4.x. All web links are clickable in PDF viewers that support hyperlinks.